大模型训练优化器全解：作用、主流算法、显存代价与选型

> **原始问题：**详细解释训练中优化器的作用以及不同主流优化器的详细讲解，并且归纳总结不同优化器的异同和适用场景

提问：2026/08/07 15:24　更新：2026/08/07 15:34　重要度：5/5

大模型训练优化器全解：作用、主流算法、显存代价与选型

一句话结论

优化器（Optimizer）的工作不是“计算梯度”，而是**把反向传播产生的瞬时梯度，变成稳定、尺度合适、带有历史信息与正则化约束的参数更新**。

```
Forward 得到 Loss
→ Backward 得到梯度 g_t
→ Unscale / Gradient Clipping
→ Optimizer 用历史状态把 g_t 变成更新方向 u_t
→ Learning-Rate Schedule 给出步长 η_t
→ 更新参数 θ_t
→ 清梯度，进入下一 Step
```

对于 Transformer/LLM：

- **AdamW 是最稳妥、最常见的默认基线**：逐参数自适应缩放，面对不同层、稀疏梯度和非平稳训练通常比纯 SGD 更容易调；
- **代价是状态显存大**：标准 AdamW 为每个参数维护一阶矩 `m` 和二阶矩 `v`；若均为 FP32，仅这两份状态就是 `8 Byte/参数`；
- **Adafactor**、**8-bit AdamW**、**Lion**主要从不同方向降低状态显存，但优化几何和调参方式并不完全等价；
- **LARS/LAMB**主要针对极大 Global Batch 的 Layer-wise 更新尺度问题；
- **Shampoo/Muon**利用矩阵结构做更强预条件或正交化，可能提高样本效率，但计算、分布式实现和调参复杂度明显更高；

- **Fused AdamW、ZeRO/FSDP Optimizer Sharding、CPU Offload 不是新的优化数学算法，**而是同一算法的系统实现与状态布局优化。

一、优化器在完整训练链路中的位置

设模型参数为：

```
θ_t: 第 t 步开始时的全部参数
```

一个 Mini-batch 的目标函数为：

```
L_t(θ_t)
```

反向传播计算：

```
g_t = ∇_θ L_t(θ_t)
```

梯度只回答：

> 在当前 Mini-batch、当前参数点附近，哪个方向让 Loss 上升最快?

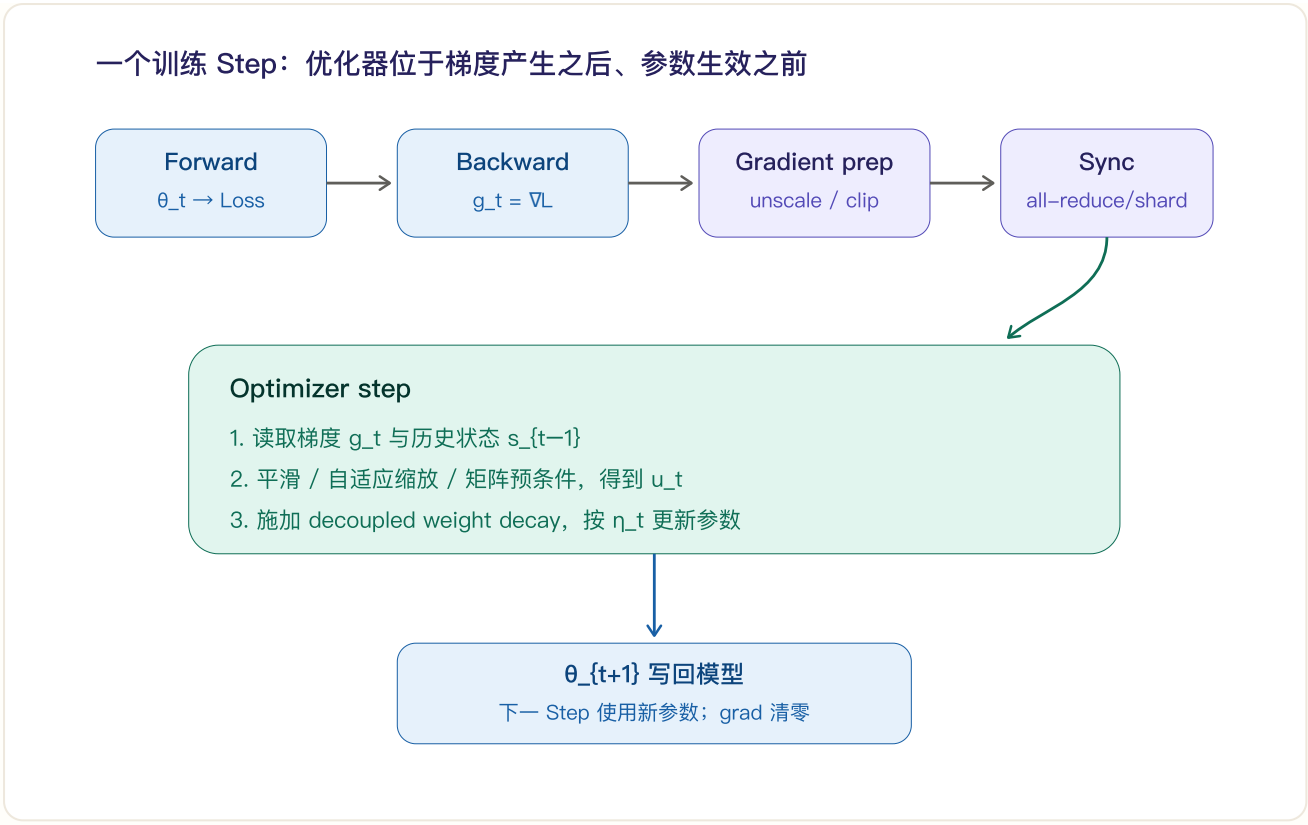
要做梯度下降，就沿相反方向移动。但真实训练中的 `g_t` 有四个问题：

1. **有噪声**：Mini-batch 梯度不是全量数据梯度；
2. **不同参数尺度差异大**：Embedding、Attention、MLP、Norm 的梯度统计不同；
3. **曲率不同**：某些方向很陡，另一些方向很平；同一个学习率可能一边震荡、一边爬得很慢；
4. **分布持续变化**：训练早期、稳定期、衰减期的梯度统计不同。

因此优化器通常实现一个状态机：

```
s_t = update_state(s_{t-1}, g_t)
u_t = transform(g_t, s_t, θ_t)
θ_{t+1} = θ_t − η_t × u_t
```

```
s_t   优化器状态，例如 Momentum、二阶矩、预条件矩阵
u_t   经过平滑、归一化或预条件后的更新方向
η_t   当前学习率，由 Scheduler 决定
```



优化器不负责什么

必须区分相邻概念：

机制	主要职责	是否属于优化器数学规则
Backward / Autograd	计算梯度 `g_t`	否
Loss Scaling	防止 FP16 梯度下溢	否，发生在梯度处理阶段
Gradient Clipping	限制异常梯度范数或数值	通常是 Step 前处理
Optimizer	将梯度及历史状态变成更新方向	是
LR Scheduler	决定每一步的全局步长 `η_t`	通常独立于优化器
Weight Decay	约束参数规模	可耦合进梯度，也可由 AdamW 解耦执行
ZeRO/FSDP	分片状态、梯度或参数	系统实现，不改变基础更新公式

二、为什么不能永远只用最朴素的 SGD

2.1 SGD

Stochastic Gradient Descent：

```
θ_{t+1} = θ_t − η_t × g_t
```

优点：

- 无额外状态；
- 更新规则透明；
- 每步计算和显存开销最低；
- 在某些视觉任务上经过充分调参可获得很好的泛化。

缺点：

- 对学习率和参数尺度敏感；
- 在狭长谷底中容易来回震荡；
- 面对 Transformer 不同模块的梯度尺度差异，需要更精细调参；
- Mini-batch 噪声直接进入更新。

2.2 Momentum SGD

一种常见记法：

```
v_t = μ × v_{t-1} + g_t
θ_{t+1} = θ_t − η_t × v_t
```

```
μ  Momentum 系数，常见量级接近 0.9
v  梯度的指数累积方向
```

不同框架可能在梯度项前乘 `1−μ`，对应状态尺度和有效学习率定义不同，但核心相同：**保留长期一致方向，抵消短期反向噪声。**

简单例子：

```
μ = 0.9
连续梯度 g = [1, 1, −0.2]

v_1 = 1
v_2 = 0.9×1 + 1 = 1.9
v_3 = 0.9×1.9 − 0.2 = 1.51
```

第三步梯度短暂变成 `-0.2`，Momentum 仍保持正向 `1.51`，不会立刻掉头。

2.3 Nesterov Momentum

Nesterov 的直觉是：

> 先按 Momentum 看一眼即将到达的位置，再根据那个位置附近的梯度修正方向。

不同库的等价公式写法较多。工程上应以框架实现为准，不要仅凭一行伪公式判断其更新顺序。

SGD 系适用场景

- CNN/视觉模型或已有成熟 SGD Recipe；
- 优化器显存极度敏感；
- 想要最简单、最容易分析的基线；
- Gradient Statistics 相对均匀，且有充足超参数搜索预算。

对于从零训练 Transformer/LLM，SGD 通常不是首选默认值，但它仍是理解其他优化器的基准坐标系。

三、从 AdaGrad、RMSProp 到 Adam：为什么要逐参数自适应

3.1 AdaGrad

为每个参数累计历史平方梯度：

```
V_t = V_{t-1} + g_t²
θ_{t+1} = θ_t − η × g_t ÷ (√V_t + ε)
```

逐元素执行。某个坐标历史梯度越大，后续有效步长越小。

优点：适合稀疏特征；出现次数少的参数不会被过度压小步长。

核心问题：`V_t` 只增不减，训练很长后有效学习率可能持续趋近 0。

3.2 RMSProp

用指数移动平均替代无限累积：

```
v_t = ρ × v_{t-1} + (1−ρ) × g_t²
θ_{t+1} = θ_t − η × g_t ÷ (√v_t + ε)
```

`ρ` 控制对历史平方梯度的遗忘速度。RMSProp 解决了 AdaGrad 的二阶统计永不遗忘问题，但没有 Adam 那套一阶矩与偏差修正的标准组合。

3.3 Adam

Adam 同时维护一阶矩和二阶矩：

```
m_t = β1 × m_{t−1} + (1−β1) × g_t
v_t = β2 × v_{t−1} + (1−β2) × g_t²
```

由于初始 `m_0=v_0=0`，早期估计偏向 0，需要 Bias Correction：

```
m_hat_t = m_t ÷ (1−β1^t)
v_hat_t = v_t ÷ (1−β2^t)
```

参数更新：

```
u_t = m_hat_t ÷ (√v_hat_t + ε)
θ_{t+1} = θ_t − η_t × u_t
```

直觉：

```
m_hat   回答“最近梯度平均朝哪里走”
v_hat   回答“这个坐标最近梯度幅度/波动多大”

m_hat ÷ √v_hat
= 方向一致性 ÷ 历史尺度
```

因此 Adam 对每个坐标形成不同的有效学习率。

3.4 一个可手算的 Adam 例子

设：

```
β1 = 0.9
β2 = 0.999
ε   暂时忽略
梯度 g_1=2, g_2=4
```

第 1 步：

```
m_1 = 0.9×0 + 0.1×2 = 0.2
v_1 = 0.999×0 + 0.001×2² = 0.004

m_hat_1 = 0.2 ÷ 0.1 = 2
v_hat_1 = 0.004 ÷ 0.001 = 4

u_1 = 2 ÷ √4 = 1
```

第 2 步：

```
m_2 = 0.9×0.2 + 0.1×4 = 0.58
v_2 = 0.999×0.004 + 0.001×16 = 0.019996

m_hat_2 = 0.58 ÷ (1−0.9²)
        ≈ 3.0526

v_hat_2 = 0.019996 ÷ (1−0.999²)
        ≈ 10.003

u_2 ≈ 3.0526 ÷ √10.003
    ≈ 0.965
```

虽然原始梯度从 2 增大到 4，经过历史尺度归一化后，更新量没有简单翻倍。这就是自适应缩放。

Adam 的适用与风险

适合：

- Transformer/LLM；
- 梯度稀疏或不同模块尺度差异大；
- 非平稳目标；
- 希望较少调参即可获得稳定基线。

风险：

- 两份逐参数状态显存大；
- `β2` 太大时二阶矩响应慢，遇到训练分布变化可能产生陈旧尺度；
- `ε` 不只是防除零，也影响小 `v` 坐标的有效步长；
- Adam 中直接加入 L2 项不等价于真正的 Weight Decay。

四、AdamW：为什么它是 LLM 默认基线

4.1 Adam 中的 L2 正则为什么有问题

如果把 L2 正则写进 Loss，梯度会变成：

```
g'_t = g_t + λ × θ_t
```

Adam 随后对整个 `g'_t` 做逐参数预条件：

```
u_t = AdamPrecondition(g_t + λθ_t)
```

于是正则项 `λθ_t` 也会被每个坐标的 `1/√v_hat` 缩放。不同参数获得不同的实际衰减强度。

对于普通 SGD，L2 正则与 Weight Decay 在合适缩放下等价；对于 Adam 这类自适应优化器，它们不等价。

4.2 AdamW 的 Decoupled Weight Decay

AdamW 将两件事分开：

```
自适应梯度更新：
θ ← θ − η_t × m_hat ÷ (√v_hat + ε)

参数衰减：
θ ← θ − η_t × λ × θ
```

合并写为：

```
θ_{t+1}
= (1−η_t λ) × θ_t
− η_t × m_hat_t ÷ (√v_hat_t + ε)
```

这样 Weight Decay 不再经过 Adam 的二阶矩预条件。

4.3 哪些参数通常不做 Weight Decay

常见 Recipe 会把参数分组：

```
Decay：Linear/Attention/MLP 的矩阵权重
No Decay：Bias、LayerNorm/RMSNorm Scale，有时还包括 Embedding
```

这是一种经验设计，不是数学定律。Embedding 是否衰减、是否对所有二维矩阵统一衰减，应以目标模型的已验证 Recipe 为准。

4.4 LLM 中的典型配套

AdamW 几乎不会单独工作：

```
AdamW
+ Warmup
+ Cosine / Linear / WSD Decay
+ Global Gradient Norm Clipping
+ BF16 或 FP16 Mixed Precision
+ FP32 Optimizer States
+ Fused Kernel
+ ZeRO/FSDP State Sharding
```

`β1/β2/ε/weight_decay/peak_lr/warmup` 是一组耦合超参数。更换模型规模、Global Batch、Token 数或精度后，不应机械复制。

五、Adafactor：怎样把二阶矩状态从 O(nm) 降到 O(n+m)

对矩阵参数：

```
W: [n,m]
G: [n,m]
```

Adam 为其保存完整二阶矩：

```
V: [n,m]
状态量 = n×m
```

Adafactor 不保存完整 `V`，而保存平方梯度统计的行、列摘要：

```
R: [n]
C: [m]
状态量 = n+m
```

再用外积近似完整二阶矩：

```
V_hat[i,j]
≈ R[i] × C[j] ÷ mean(R)
```

随后：

```
U = G ÷ (√V_hat + ε)
```

原论文还强调：

- Update Clipping：防止二阶矩衰减过慢时产生过大的更新；
- 随训练步数变化的二阶矩衰减率；
- Relative Step Size：根据参数 RMS 调整更新尺度；
- 可不使用一阶 Momentum，从而进一步减少状态。

数量级例子

假设矩阵：

```
W: [4096,16384]
元素数 = 67,108,864
```

FP32 完整二阶矩：

```
67,108,864 × 4 Byte
= 268,435,456 Byte
= 256 MiB
```

Adafactor 行列状态：

```
(4096 + 16384) × 4 Byte
= 81,920 Byte
≈ 80 KiB
```

这里只比较该矩阵的二阶矩，不包含一阶 Momentum、参数、梯度和临时 Buffer。

Adafactor 的边界

- 对二维大矩阵节省非常显著；
- 向量、小矩阵无法获得同等级因式分解收益；
- 因式分解是假设性近似，不等价于完整逐元素二阶矩；
- 不同库对 Relative Step、Warmup、Scale Parameter、Momentum 的默认值差异大；
- 从 AdamW 切换时不能直接照搬学习率。

适合：优化器状态成为硬瓶颈、T5/Transformer 类模型、参数矩阵很大，且愿意重新验证稳定性和超参数。

六、LARS 与 LAMB：极大 Batch 下的 Layer-wise Trust Ratio

6.1 为什么极大 Batch 会遇到更新尺度问题

Global Batch 增大后，梯度噪声下降，通常希望提高学习率以增加吞吐。但不同层的：

```
参数范数 ||θ_l||
梯度/更新范数 ||u_l||
```

差异可能很大。一个全局学习率可能让某些层更新过猛，另一些层几乎不动。

6.2 LARS

LARS 主要建立在 SGD/Momentum 更新上，按层计算 Trust Ratio：

```
r_l = ||θ_l|| ÷ (||g_l|| + λ||θ_l|| + ε)
```

更新近似：

```
θ_l ← θ_l − η_t × r_l × (g_l + λθ_l)
```

它在极大 Batch 的视觉训练中影响较大，但并不是 Transformer 的通用默认选择。

6.3 LAMB

LAMB 先得到 Adam 类方向：

```
a_l = m_hat_l ÷ (√v_hat_l + ε)
r_l = a_l + λθ_l
```

再计算 Layer-wise Trust Ratio：

```
trust_l = ||θ_l|| ÷ (||r_l|| + ε)
```

更新：

```
θ_l ← θ_l − η_t × trust_l × r_l
```

因此 LAMB 可以理解为：

```
Adam 的逐参数自适应
+ Layer-wise Update Norm 控制
```

LAMB 原始工作重点是极大 Batch 的 BERT/ResNet 训练。它解决的不是“模型太大放不下”，而是“大 Batch 提高学习率后，各层相对更新尺度失衡”。

适合：Global Batch 极大、已有 LAMB Recipe、希望保持 Layer-wise 更新比例。普通规模训练不应仅因为“模型大”就自动选择 LAMB。

七、Lion：用 Sign Momentum 换掉二阶矩

Lion 只维护一份 Momentum 状态：

```
c_t = β1 × m_{t−1} + (1−β1) × g_t
```

参数更新使用符号：

```
θ_t = θ_{t−1} − η_t × (sign(c_t) + λθ_{t−1})
```

再更新状态：

```
m_t = β2 × m_{t−1} + (1−β2) × g_t
```

核心差异：

```
AdamW: m_hat ÷ √v_hat，连续地使用梯度幅度
Lion: sign(c)，每个坐标主要保留方向，幅度约为全局学习率
```

优点：

- 只需一份状态，FP32 状态约 `4 Byte/参数`；
- 更新计算简单；
- 某些训练任务中可获得有竞争力的结果。

代价：

- 丢弃了逐坐标更新幅度；
- 学习率直接控制每个坐标的更新大小，通常比 AdamW 更敏感；
- Weight Decay 与学习率需要重新搜索；
- AdamW 的超参数不能原样迁移。

适合：状态显存重要、愿意做独立 LR/WD/Beta Sweep，并已有目标模型上的验证。若只允许一次高成本预训练，AdamW 往往仍是更保守的选择。

八、Shampoo：从逐元素缩放升级为矩阵预条件

Adam 的二阶矩是对角预条件：每个参数独立缩放，不建模同一矩阵不同行列之间的相关性。

对于二维梯度：

```
G_t: [n,m]
```

Shampoo 累计两个矩阵统计：

```
L_t = L_{t-1} + G_t G_t^T      [n,n]
R_t = R_{t-1} + G_t^T G_t      [m,m]
```

预条件后的梯度近似：

```
G_pre = L_t^(-1/4) × G_t × R_t^(-1/4)
```

为什么是 `-1/4`：左右各承担一部分矩阵二阶预条件，使组合效果对应更完整的曲率归一化。

优点：

- 利用参数 Tensor 的矩阵结构；
- 比 Adam 的对角缩放表达更丰富；
- 可能用更少 Step 达到目标 Loss。

代价：

- 状态从 `O(nm)` 变成 `O(n²+m²)`，对非常不方正的矩阵可能有利，但大维度仍可能很重；
- 需要计算逆矩阵根；
- 常通过 Block Partition、低频更新 Preconditioner、分布式 Preconditioner 等方式控制成本；
- 每步墙钟时间、通信和实现复杂度更高。

适合：训练成本由“需要多少 Step”主导，愿意以更复杂的单步计算换样本效率，并有成熟分布式实现。它不是随手替换 AdamW 的低风险选项。

九、Muon：矩阵 Momentum 的近似正交化

Muon 是面向二维矩阵参数的新兴方案。基本思路：

```
M_t = μ × M_{t-1} + G_t
O_t = Orthogonalize(M_t)
W_t = W_{t-1} − η_t × O_t
```

实践中常用 Newton–Schulz Iteration 近似计算矩阵的极分解方向，而不是精确 SVD。

直觉：

> AdamW 对每个元素单独缩放；Muon 把整个梯度矩阵视作一个线性映射，调整其奇异方向，使更新在矩阵空间中更均衡。

工程上通常是 Hybrid Optimizer：

```
二维 Hidden Weight Matrices → Muon
Embedding / LM Head / Bias / Norm / Scalar → AdamW
```

原因是 Muon 的正交化天然针对矩阵，向量和特殊参数不适用同一规则。

截至 2026-08，已有大规模 LLM/MoE 技术报告展示了分布式 Muon 实践，但应注意：

- 报告中的计算效率收益依赖模型、数据、Recipe 和对照调参质量，不能视为所有任务的固定倍数；
- Newton–Schulz 增加额外矩阵运算；
- FSDP/ZeRO 将矩阵展平或切片后，如何恢复完整矩阵语义并高效通信是系统难点；
- 更新尺度、Weight Decay 和矩阵长宽比需要专门处理；
- 生态成熟度和可复现实证仍弱于 AdamW。

适合：愿意投入研究工程预算、追求更高样本效率，并能进行严格 AdamW 对照实验的大规模预训练团队。默认生产基线仍应保留 AdamW。

十、8-bit AdamW、Fused AdamW、ZeRO：不要把实现优化误认成新算法

10.1 8-bit AdamW

核心更新仍是 AdamW，但 `m/v` 使用 8-bit 或块量化格式保存，更新时解量化、计算、再量化。

收益：显著减少状态显存和状态读写带宽。

风险：

- Quantization Scale/Block Size 影响误差；
- 小 Tensor、异常值和非常小的状态需要特殊处理；
- 并不等价于把 AdamW 的数学状态精确保存为 FP32。

如果目标是“尽量保留 AdamW 行为，同时降低显存”，8-bit AdamW 通常比直接换优化器家族更接近原基线。

10.2 Fused / Foreach AdamW

把大量逐 Tensor 操作合并成少量 GPU Kernel：

```
读取参数、梯度、m、v
→ 更新 m/v
→ Bias Correction
→ 参数更新与 Weight Decay
```

数学规则可相同，但减少 Kernel Launch 和 HBM 往返。Fused 是性能实现属性，不代表优化效果一定不同。

10.3 ZeRO-1 / FSDP Optimizer State Sharding

DDP 中每个 Rank 通常保留完整参数和完整优化器状态，状态存在 N 份副本。

ZeRO-1 将 Optimizer States 按 DP Rank 分片：

```
每个 Rank 只更新自己负责的参数分片
→ 更新后同步新参数分片
```

FSDP/ZeRO-2/3 还可以进一步分片梯度和参数。它们改变的是状态放在哪里、何时通信，不应改变理想数学更新。

十一、优化器状态到底占多少显存

假设：

```
参数量 P
BF16 参数       2 Byte/参数
BF16 梯度       2 Byte/参数
FP32 Master     4 Byte/参数
```

```
FP32 Momentum m 4 Byte/参数
FP32 Variance v 4 Byte/参数
```

典型 Mixed-Precision AdamW 静态模型状态近似：

```
2 + 2 + 4 + 4 + 4
= 16 Byte/参数
```

这不包含 Activation、临时 Buffer、通信 Bucket、Allocator 碎片和 Workspace。

7B 参数数值例子

```
P = 7×10⁹
```

AdamW 两份 FP32 Moments：

```
7×10⁹ × 8 Byte
= 56×10⁹ Byte
≈ 52.2 GiB
```

加 FP32 Master Weight：

```
7×10⁹ × 12 Byte
= 84×10⁹ Byte
≈ 78.2 GiB
```

再加 BF16 参数和梯度：

```
7×10⁹ × 16 Byte
= 112×10⁹ Byte
≈ 104.3 GiB
```

量纲检查：

```
参数 × Byte/参数 = Byte
GiB = Byte ÷ 2³⁰
```

若 8 个 DP Ranks 理想均匀分片全部这些状态：

```
104.3 GiB ÷ 8 ≈ 13.0 GiB/Rank
```

真实峰值还取决于参数 All-Gather、梯度 Reduce-Scatter、Prefetch、Checkpoint 和临时 Full Buffer。

状态显存对比

优化器	主要状态	FP32 额外状态量级	特点
SGD	无	0 B/参数	最省状态
Momentum SGD	`v`	4 B/参数	一阶平滑
Adam/AdamW	`m,v`	8 B/参数	一阶+逐参数二阶矩
Lion	`m`	4 B/参数	Sign Momentum
8-bit AdamW	量化 `m,v`	通常约 2 B/参数量级，另有 Scale/元数据	行为接近 AdamW，但有量化误差
Adafactor	行/列二阶矩，可选 Momentum	大矩阵二阶状态约 O(n+m)	节省取决于 Shape
LAMB	`m,v` + 临时范数	约 8 B/参数	Adam 类状态，加 Layer Trust Ratio
Shampoo	每维 Preconditioner	依 Shape/Block 而变	状态和计算不再按固定 B/参数描述
Muon	矩阵 Momentum；非矩阵常另用 AdamW	主矩阵约 4 B/参数，混合部分另算	额外正交化计算

十二、统一对比：这些优化器到底有什么相同和不同

优化器差异的三条轴：历史状态、缩放粒度、结构信息

输入都来自同一个 g_t；最终都产生参数更新 θ_{t+1}

差别在于如何过滤噪声、归一化尺度，以及是否利用 Layer/Matrix 结构

梯度与一阶历史

SGD：raw gradient

Momentum：EMA direction

Lion：sign(momentum)

逐元素自适应

AdaGrad / RMSProp

AdamW：m / $\sqrt{v}$

Adafactor：factored v

Layer / Matrix 结构

LARS/LAMB：trust ratio

Shampoo：matrix roots

Muon：orthogonalization

选型不是只看收敛曲线

算法轴：样本效率、稳定性、超参数敏感度、正则化行为

系统轴：状态显存、HBM 流量、额外 FLOP、通信、Checkpoint 体积

运营轴：实现成熟度、失败恢复、可观测性、Recipe 可迁移性

公平比较必须分别调 LR / WD / Warmup / Betas，不能照搬 AdamW 参数。

优化器	更新几何	状态成本	调参难度	典型优势	主要风险
SGD	原始梯度	最低	高	简单、低显存	对尺度和 LR 敏感
Momentum SGD	一阶平滑	低	中	降噪、加速一致方向	无逐参数自适应
AdamW	一阶矩/二阶矩逐元素预条件	高	中低	Transformer 稳定默认	状态显存和 HBM 流量大
Adafactor	因式分解二阶矩	低到中	中高	大矩阵状态极省	近似误差、实现默认值差异
LARS	SGD + Layer Trust Ratio	中低	高	极大 Batch 视觉训练	Transformer 不一定适合
LAMB	Adam + Layer Trust Ratio	高	高	极大 Batch Transformer/BERT	普通 Batch 收益不确定
Lion	Sign Momentum	中低	高	状态约为 AdamW 一半	LR/WD 敏感、丢失幅度信息
Shampoo	矩阵预条件	Shape 相关	很高	可能减少所需 Steps	逆根、状态和分布式复杂
Muon	矩阵 Momentum 正交化	混合	很高	新兴的样本效率潜力	生态和大规模复现尚在发展

所有优化器的共同点

- 都依赖同一个目标函数和 Backward 梯度；
- 都需要 Learning-Rate Schedule；
- 都可能被异常梯度、错误 Loss Scaling、数据问题破坏；
- 都必须与模型规模、Batch、精度和并行方式共同调优；
- 都需要保存状态以便 Checkpoint 精确续训，只有无状态 SGD 例外。

最根本的不同点

1. **是否记忆历史**：SGD 不记；Momentum/Lion 一阶；AdamW 一阶+对角二阶；
2. **缩放粒度**：全局、逐层、逐参数、逐矩阵；
3. **是否使用梯度幅度**：Lion 主要用 Sign，AdamW 使用连续幅度；
4. **状态精度与布局**：FP32、8-bit、Factored、Sharded、Offloaded；
5. **每步成本与所需 Step 数的权衡**：复杂优化器可能单步更贵但目标 Loss 所需 Step 更少。

十三、按场景给出选型建议

场景 A：第一次训练新的 Transformer/LLM

推荐：

```
AdamW + BF16 + FP32 States
+ Gradient Clipping
+ Warmup + 已验证的 LR Schedule
+ Fused Implementation
```

理由：资料、实现、调参经验和故障诊断最成熟。先建立可信基线，再比较其他优化器。

场景 B：AdamW 状态显存成为硬瓶颈

优先级通常是：

1. ZeRO-1/FSDP Shard Optimizer States；
2. 若通信/集群允许，继续优化 Sharding/Offload；
3. 8-bit AdamW，尽量保持 AdamW 更新行为；

4. Adafactor 或 Lion，但必须重新调参和验证质量。

原因：先改变状态布局，通常比直接更换优化算法风险低。

场景 C：极大 Global Batch，扩展后训练不稳定或效果下降

- 先检查 LR Scaling、Warmup、Gradient Noise Scale、数据重复和有效 Token Batch；
- Transformer/BERT 类可评估 LAMB；
- 视觉 CNN 可评估 LARS；
- 不要把 LAMB 当作任意大模型的默认选项。

场景 D：研究更高样本效率

可评估 Shampoo/Muon/SOAP/Sophia 等预条件或二阶近似方向，但公平实验必须：

```
同模型、同数据顺序、同 Token Budget
分别调最优 LR/WD/Warmup
同时报告 Validation Loss vs Tokens
和 Validation Loss vs Wall-clock / FLOPs
```

只报告“达到某 Loss 的 Step 更少”不够，因为每步可能更慢、通信更多。

场景 E：LoRA/PEFT 微调

可训练参数很少时，AdamW 的状态显存通常不再是主矛盾。优先选择稳定简单的 AdamW；只有在全参数微调或超大 Adapter 时，8-bit State 才更重要。

场景 F：Sparse Embedding 或极稀疏参数

AdaGrad 类方法仍可能有价值，因为低频坐标保留更大的有效学习率。具体需看框架是否支持稀疏梯度和目标参数类型。

十四、最小 PyTorch 示例：正确组织参数组和训练 Step

```
import torch
import torch.nn as nn

torch.manual_seed(0)
model = nn.Sequential(
    nn.Linear(8, 16),
    nn.LayerNorm(16),
    nn.GELU(),
```

```
    nn.Linear(16, 4),
).cuda().to(torch.bfloat16)

decay, no_decay = [], []
for name, p in model.named_parameters():
    if not p.requires_grad:
        continue
    if p.ndim >= 2 and "norm" not in name.lower():
        decay.append(p)
    else:
        no_decay.append(p)

optimizer = torch.optim.AdamW(
    [
        {"params": decay, "weight_decay": 0.1},
        {"params": no_decay, "weight_decay": 0.0},
    ],
    lr=3e-4,
    betas=(0.9, 0.95),
    eps=1e-8,
)

for step in range(10):
    x = torch.randn(32, 8, device="cuda", dtype=torch.bfloat16)
    target = torch.randn(32, 4, device="cuda", dtype=torch.bfloat16)

    optimizer.zero_grad(set_to_none=True)
    pred = model(x)
    loss = ((pred - target) ** 2).mean()
    loss.backward()

    grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    optimizer.step()

    print(step, float(loss), float(grad_norm))
```

说明：

- 示例用 BF16，所以通常不需要 FP16 GradScaler；具体硬件与算子仍需验证；
- `set_to_none=True` 避免无意义清零写带宽；
- `p.ndim>=2` 是常见参数分组启发式，不是所有模型的绝对规则；
- 生产环境还应加入 LR Scheduler、Gradient Accumulation、Distributed Synchronization、Checkpoint 和 NaN/Inf 检查；
- `betas=(0.9,0.95)` 只是 LLM 中常见的一类配置，不是通用最优值。

十五、训练中应该监控什么

15.1 优化动态

```
Global Grad Norm
Update Norm
Parameter Norm
Update-to-Parameter Ratio = ||Δθ|| ÷ ||θ||
```

```
每层的 Grad/Update/Parameter Norm
Adam m/v 的 RMS 与最大值
Gradient Clipping 触发比例
NaN/Inf 与 Loss Scale
```

15.2 系统性能

```
optimizer.step wall time
optimizer kernel 数量与 launch 间隙
HBM read/write throughput
Fused kernel 是否生效
Optimizer State 总显存
State Sharding 的通信时间
Checkpoint 保存/加载耗时与体积
CPU Offload 的 PCIe/NVLink-C2C 流量
```

15.3 质量与公平比较

```
Training Loss vs Tokens
Validation Loss vs Tokens
Validation Loss vs Wall-clock
Validation Loss vs Training FLOPs
下游任务指标
不同随机种子的稳定性
```

若一个优化器每 Step 更快但需要更多 Steps，或者每 Step 更慢但样本效率更高，只看单一维度都会得出错误结论。

十六、常见误区

误区 1：优化器负责反向传播

错误。Backward 计算梯度；Optimizer 消费梯度并更新参数。

误区 2：Adam 的 L2 正则就是 AdamW

错误。Adam 中把 $\lambda\theta$ 加入梯度后会被二阶矩预条件；AdamW 把 Weight Decay 与自适应梯度更新解耦。

误区 3：状态更少的优化器一定更快

错误。Muon/Shampoo 虽状态结构可能有优势，但额外矩阵运算可能提高每步成本。8-bit State 还需要量化/解量化。

误区 4：Fused AdamW 是新的优化算法

错误。它通常是相同更新公式的 Kernel Fusion 实现。

误区 5：ZeRO 会改变 AdamW 的数学结果

理想情况下不会；它改变状态、梯度和参数的布局及通信。精度、Reduction Order、Offload 和异步执行可能产生微小数值差异，但不是新的优化器规则。

误区 6：换优化器可以直接复用 AdamW 的学习率

错误。Lion、Adafactor、LAMB、Muon 的更新尺度完全不同，必须独立调 LR、WD、Warmup 和内部系数。

误区 7：训练 Loss 降得最快就是最好优化器

错误。还要看 Validation、最终能力、Wall-clock、FLOPs、状态显存、失败率和调参成本。

十七、决策树式总结

```
是否缺少可信基线?
└ 是 → AdamW

AdamW 是否主要卡在状态显存?
├ 可以做分片 → ZeRO-1/FSDP Optimizer Sharding
├ 希望最小改变算法 → 8-bit AdamW
├ 大矩阵为主且愿意调参 → Adafactor
└ 可接受改变更新几何 → Lion

是否是极大 Global Batch?
├ Transformer/BERT 类 → 评估 LAMB
└ 视觉 CNN → 评估 LARS

是否有研究预算追求更少训练 Tokens/Steps?
├ 成熟矩阵预条件实现 → Shampoo
├ 二维权重为主、可做 Hybrid → Muon
└ 否 → 保持 AdamW，优化数据、Schedule 和系统实现
```

最重要的工程原则：

> **先把 AdamW 基线做到可信，再改变一个维度。优化器比较必须同时看算法质量和系统成本。**

可选自测

1. 数值题

一个 7B 模型使用 BF16 参数、BF16 梯度、FP32 Master Weight 和 FP32 AdamW `m/v`。不考虑 Activation 和临时 Buffer，为什么静态状态约为 `16 Byte/参数`？8 个 Ranks 完全分片后，每 Rank 理论值是多少?

2. 因果题

为什么把 `λθ` 直接加进 Adam 梯度，不等价于 AdamW 的 Decoupled Weight Decay?

3. 工程题

一个新 LLM 使用 AdamW 时 OOM，但训练质量和稳定性都很好。你会按照什么顺序评估 ZeRO/FSDP、8-bit AdamW、Adafactor 和 Lion？为什么?

继续学习

1. 前置：Learning-Rate Schedule 与 Warmup —— 优化器方向如何与全局步长共同决定稳定性。
2. 同层对比：AdamW、Muon 与 Shampoo 的矩阵更新几何 —— 对角预条件、正交化和矩阵逆根的本质差异。
3. 下游应用：ZeRO/FSDP Optimizer State Sharding —— 优化器状态如何在多 GPU 上分片、通信与恢复。

回复 1、2 或 3 继续；也可以说"稍后"或"不感兴趣"。

事实来源与版本边界

截至 2026-08-07，本讲解依据稳定算法原理及原始资料，包括：Adam（Kingma & Ba, 2015）、Decoupled Weight Decay/AdamW（Loshchilov & Hutter, 2019）、Adafactor（Shazeer & Stern, 2018）、LAMB（You et al., 2020）、Shampoo（Gupta et al., 2018）、Lion（Chen et al., 2023）及 Muon 大规模 LLM 技术报告（Liu et al., 2025）。

具体框架的公式约定、Bias Correction 顺序、Weight Decay、Fused Kernel、State Precision、默认超参数和分布式布局可能随 PyTorch、JAX、TensorFlow、DeepSpeed、Megatron Core 或目标训练栈版本变化。生产配置应以目标版本源码、Checkpoint 格式和实测为准；论文报告中的效率

收益不能跨模型、数据、Token Budget、硬件和调参预算直接外推。